

ProductionManagementAI — 完了の記録とダッシュボードのクエリ — テーブル定義書

003_DB — 要件 REQ-028～REQ-039（work-items/WI-004/brief.md）、基本設計 003_BD（docs/en/010_basic-design/003_BD_ダッシュボード.md）。003_DD と 003_DD-API（docs/en/020_detailed-design/、作成予定）と、001_DD への REQ-033 の修正を実装する。

本書は 001_DB が定義した 1 つのテーブルを変更する：production_orders に列 1 つとチェック制約 2 つ、部分インデックス 2 つ、デモ用の履歴を追加する。001_DB と 002_DB が products、production_orders、production_order_number_counters について述べていることはすべて有効なままで、再掲しない。

物理名は 000_DB～002_DB に従う：snake_case、UTC のタイムスタンプは接尾辞 _utc の timestamptz、EF Core の pk_ / fk_ / ix_ / ck_ の制約名。

改版履歴

版	日付	作成者	改版内容
1	2026-09-22	Claude（ThanhTN 向け）	新規作成（WI-004 DEC-003、DEC-004、DEC-013～DEC-015）

テーブル一覧

テーブル	物理名	目的	003_DB での変更
製品	products	製品の参照データ（001_DB）	なし — 読み取りのみ。製品のラベルのために結合
製造指示	production_orders	製造指示 1 件につき 1 行（001_DB）	新しい列 completed_at_utc、チェック制約 2 つ、部分インデックス 2 つ、補完、デモ用の履歴
指示番号カウンター	production_order_number_counters	年ごとの指示番号の連番（001_DB）	初期データの年のカウンターを新しいデモ用の製造指示の後まで進める

ER 図とリレーション

001_DB から変更なし。ダッシュボードがたどるリレーションは 1 つ：

テーブル	関連テーブル	リレーション（1:1 / 1:N / N:N）	FK 列
products	production_orders	1:N	production_orders.product_id

テーブル定義

production_orders — 追加する列

項目名	物理名	データ型	長さ	NOT NULL	既定値	PK/FK	備考
CompletedAtUtc	completed_at_utc	timestampzt	—	no	なし	—	製造指示が Completed になった時刻（REQ-033、DEC-003）。製造指示を InProgress から Completed にする保存の中で、その保存の UTC 時刻 — updated_at_utc に書き込むのと同じ値 — をアプリが設定する。その後は変更せず、リクエストから受け取らない。他のすべてのステータスでは NULL。既存の完了済みの製造指示は補完する（マイグレーションの影響を参照）

履歴テーブルではなく列にする理由： Completed には InProgress からしか到達できず終端状態なので（001_BD M-02）、製造指示が完了するのは多くて 1 回で、1 つのタイムスタンプで事実全体を表せる。DEC-003 がそのトレードオフを記録している。

ダッシュボードが読む列

物理名	用途
status	ステータスのタイル（REQ-029）。「未完了」= Draft または InProgress
due_date	納期遅れ／納期間近のグループ（D-01、D-02）、作業量の区分（D-03）、期限内の比較（D-07）
quantity	作業量と上位の製品の数量（D-03、D-04）、完了した数量（D-05、D-06）
product_id	上位の製品のグループ化（D-04）。ラベルのための products との結合
created_at_utc	リードタイム（D-09）
completed_at_utc	すべての納品実績の指標（D-05～D-09）
id、order_number	表示する最大 20 件の製造指示（D-01、D-02）。同順位の解決

notes、updated_at_utc、order_year、order_seq、xmin は読まない。

制約

制約	種類 (PK/FK/UNIQUE/CHECK)	テーブル列	参照先	ルール
ck_production_orders_completed_at_matches_status	CHECK	production_orders.status、 completed_at_utc	—	(status = 'Completed') = (completed_at_utc IS NOT NULL) — 完了した製造指示は必ず完了日時を持ち、他の製造指示は持たない (REQ-033 の失敗基準)
ck_production_orders_completed_at_not_before_created	CHECK	production_orders.completed_at_utc、 created_at_utc	—	completed_at_utc IS NULL OR completed_at_utc >= created_at_utc — 何が行を書き込んでも、リードタイムが負になることはない (REQ-037 の失敗基準)

001_DB がアプリに任せるルール（納期 ≥ 今日、遷移のグラフ）と違い、この 2 つは時計に関係なく常に成り立つため、データベースで強制できる。これらはアプリのルールの下での安全網で、それを置き換えるものではない：値はエンティティが設定し（001_DD モジュール 1、修正済み）、設定し忘れたコードの経路は、納品実績の数値から漏れた製造指示を黙って作るのではなく、保存がはっきり失敗する。

遷移のルール自体（InProgress → Completed のみ、Completed は終端）は、001_DB が決めたとおりアプリで強制する。Completed は終端状態なので、1 つ目の制約は、正しい遷移で completed_at_utc がクリアされることはないという意味にもなる。

アクセスパターン

ダッシュボードは読み込みごとに 7 つの文（Q1～Q6、Q3 は要注意のグループごとに 1 回）を発行し、1 つのスナップショットを見るよう、すべてを 1 つの読み取り専用の REPEATABLE READ トランザクションの中で実行する（DEC-015。トランザクションと同時実行制御を参照）。日付に依存する値はすべてアプリが工場の時計から計算し、パラメータとして渡す。SQL がタイムスタンプを工場ローカルの日付に変換する必要がある箇所では、工場のタイムゾーン名自体もパラメータとして渡す（@zone、PlantOptions.TimeZone から）。クライアントからの入力はこの文にも届かない — エンドポイントはパラメータを持たない（003_BD §5）。

パラメータ。すべてスナップショットごとに 1 回、IPlantClock から計算する（T = 工場の今日、W = T の週の月曜日）：

パラメータ	値	使用するクエリ
@today	T（日付）	Q2～Q4
@soonEnd	T + 7（日付）	Q3（納期間近）

パラメータ	値	使用するクエリ
@week0	W(T) (日付)	Q2 (作業量の区分)
@trendStartUtc	工場ローカルの日 W(T) - 77 の開始、UTC の時刻として	Q5、Q6 の下限
@weekStartUtc	工場ローカルの日 W(T) の開始、UTC	Q5 (今週の完了)
@monthStartUtc	T の月の工場ローカルの日 1 の開始、UTC	Q5 (今月の完了)
@windowStartUtc	工場ローカルの日 T - 29 の開始、UTC	Q5 (期限内、リードタイム)
@endUtc	工場ローカルの日 T + 1 の開始、UTC	Q5、Q6 の上限
@zone	Asia/Tokyo (設定値)	Q5、Q6 の工場の日付への変換

@trendStartUtc は常に最も早い下限である：推移は少なくとも 77 日さかのぼり、月と 30 日の期間はさかのぼっても 30 日。範囲の述語は completed_at_utc 自体に対して UTC の時刻と比較する形で書くので、下のインデックスが使える。工場の日付への変換はグループ化と期限内の比較にだけ現れ、すでに絞り込んだ行に対して実行される。

Q1 — ステータス別件数 (REQ-029)

```
SELECT status, count(*) FROM production_orders GROUP BY status;
```

Q2 — 作業量の区分 (D-03、REQ-031)、未完了の製造指示のみ

```
SELECT CASE
    WHEN due_date < @today          THEN -1          -- Overdue
    WHEN due_date >= @week0 + 56    THEN  8          -- Later
    ELSE (due_date - @week0) / 7    -- week 0..7
END AS bucket,
count(*), sum(quantity)
FROM production_orders
WHERE status IN ('Draft', 'InProgress')
GROUP BY bucket;
```

due_date - @week0 は日数の整数なので、整数の除算で T から W(T) + 55 までのすべての日付が第 0～7 週のちょうど 1 つに入る。行のない区分はアプリが 0 で埋める (REQ-031)。納期遅れの区分の件数は D-01 の合計でもあり、Q3 が独立に読む — どちらも同じスナップショットから来るので一致する。

Q3 — 要注意のグループ（D-01、D-02、REQ-030）、範囲を変えて 2 回実行

```
SELECT o.id, o.order_number, o.quantity, o.due_date, o.status, p.sku, p.name,  
       count(*) OVER () AS total  
FROM production_orders o  
JOIN products p ON p.id = o.product_id  
WHERE o.status IN ('Draft', 'InProgress')  
      AND o.due_date < @today                -- overdue  
      -- or: AND o.due_date BETWEEN @today AND @soonEnd    -- due soon  
ORDER BY o.due_date, o.order_number  
LIMIT 10;
```

`count(*) OVER ()` は `LIMIT` より前に評価されるので、1 つの文で先頭 10 行とグループの合計の両方を返す。グループが空なら行を返さず、合計は 0。（下では Q3a と Q3b として挙げる。）

Q4 — 上位の製品（D-04、REQ-032）

```
SELECT p.id, p.sku, p.name, sum(o.quantity) AS open_quantity, count(*) AS active_orders  
FROM production_orders o  
JOIN products p ON p.id = o.product_id  
WHERE o.status IN ('Draft', 'InProgress')  
GROUP BY p.id, p.sku, p.name  
ORDER BY open_quantity DESC, p.sku  
LIMIT 10;
```

Q5 — 納品実績の数値（D-05、D-06、D-07、D-09、REQ-034、REQ-035、REQ-037）、1 行

```

SELECT
  count(*)      FILTER (WHERE completed_at_utc >= @weekStartUtc) AS week_count,
  sum(quantity) FILTER (WHERE completed_at_utc >= @weekStartUtc) AS week_quantity,
  count(*)      FILTER (WHERE completed_at_utc >= @monthStartUtc) AS month_count,
  sum(quantity) FILTER (WHERE completed_at_utc >= @monthStartUtc) AS month_quantity,
  count(*)      FILTER (WHERE completed_at_utc >= @windowStartUtc) AS window_count,      -- y
  count(*)      FILTER (WHERE completed_at_utc >= @windowStartUtc
                        AND (completed_at_utc AT TIME ZONE @zone)::date <= due_date)
                        AS window_on_time,      -- x
  avg(extract(epoch FROM completed_at_utc - created_at_utc) / 86400.0)
  FILTER (WHERE completed_at_utc >= @windowStartUtc) AS window_lead_days
FROM production_orders
WHERE completed_at_utc >= @trendStartUtc
  AND completed_at_utc < @endUtc;

```

行がないときの `sum` と `avg` は `NULL` を返す。アプリは `NULL` の合計を 0 に、(`window_count = 0` での) `NULL` の平均を "—" に対応付ける (003_BD M-13、M-14)。丸めと表示はアプリの役割なので、データベースは丸めない平均を返す。

Q6 — 完了の推移 (D-08、REQ-036)

```

SELECT date_trunc('week', completed_at_utc AT TIME ZONE @zone)::date AS week_start, count(*)
FROM production_orders
WHERE completed_at_utc >= @trendStartUtc
  AND completed_at_utc < @endUtc
GROUP BY week_start;

```

PostgreSQL の `date_trunc('week', ...)` は ISO 週の月曜日に切り詰めるので、ちょうど `W(d)` になる (DEC-008)。アプリは結果を 12 個の月曜日 `W(T) - 77、...、W(T)` に並べ、欠けた週を 0 で埋める。

`@endUtc` は Q5 と Q6 を制限し、スナップショットの今日より後の完了のタイムスタンプが数値に入らないようにする。本番では起こりえないが、データベースの `now()` とは独立に工場の時計を固定するテストでは意味がある。

アクセスパターン	頻度	対応するもの
Q1 ステータス別件数	ダッシュボードの読み込みごとに 1 回	<code>production_orders</code> のシーケンシャルスキャン — 性能を参照
Q2、Q4 未完了の製造指示の集計	読み込みごとにそれぞれ 1 回	<code>ix_production_orders_active_due_date</code> （未完了の集合のビットマップスキャン）
Q3a 納期遅れ、Q3b 納期間近：納期順の先頭 10 件	読み込みごとにそれぞれ 1 回	インデックス順での <code>ix_production_orders_active_due_date</code>
Q5、Q6 最近の完了	読み込みごとにそれぞれ 1 回	<code>ix_production_orders_completed_at_utc</code> の範囲スキャン

インデックス定義

001_DB と 002_DB の既存のインデックスは変更しない。003_DB で新規：

インデックス名	テーブル	列	種類	根拠（アクセスパターン）
<code>ix_production_orders_active_due_date</code>	<code>production_orders</code>	<code>due_date,</code> <code>order_number</code> WHERE <code>status IN ('Draft',</code> <code>'InProgress')</code>	btree、部分	Q2～Q4 は未完了の製造指示だけを読む。未完了の製造指示は未完了の作業量で、 <code>Completed</code> と <code>Cancelled</code> の製造指示がいつまでも増え続ける間もおおむね一定なので、時間とともに未完了の集合はテーブルの小さな一部になる。部分インデックスにより、これらのクエリはテーブルの履歴ではなく未完了の作業量に比例する。 <code>(due_date, order_number)</code> の順はちょうど Q3 の <code>ORDER BY</code> なので、各要注意のグループは順に読まれ、10 行で止まる。002_DB の全体の <code>ix_production_orders_due_date_order_number</code> でも Q3 は扱えるが、最初の納期遅れの未完了の製造指示に届く前に、納期の古い完了・取消済みの製造指示をすべてたどってしまう
<code>ix_production_orders_completed_at_utc</code>	<code>production_orders</code>	<code>completed_at_utc</code> WHERE <code>completed_at_utc IS</code> <code>NOT NULL</code>	btree、部分	Q5 と Q6 は、これまでに記録されたすべての完了のうち、直近約 12 週の完了を読む。このインデックスの範囲スキャンはその期間だけを読む。部分の述語は、そうでなければすべて <code>NULL</code> の項目になる未完了の行をすべて除く

意図的に追加しなかったインデックス

候補	追加しない理由
<code>status</code> （Q1 用）	Q1 はすべての行のステータスを必要とするので、インデックスが役立つのはインデックスオンリースキャンの場合だけで、一日中更新されるテーブルの可視性マップではそれは当てにならない。性能に示す規模では、テーブルのスキャンは安価である。およそ 100 万行を超えたら見直す。そのときの答えはインデックスではなく、維持するステータスごとのカウンターである
<code>(product_id) WHERE active</code> （Q4 用）	Q4 はどのインデックスで見つけても未完了の集合全体を集計し、製品ごとのグループ化はその集合に対するハッシュ集計である。未完了の部分インデックスがすでに読む行を限定している

候補	追加しない理由
(completed_at_utc) INCLUDE (quantity, due_date, created_at_utc) のカバリングインデックス	Q5/Q6 をインデックスオンリーにできるが、一日中更新されるテーブルのインデックスを広げて、最大約 12 週分の完了のヒープの読み取りを省くだけである。この件数では見合わない

トランザクションと同時実行制御

操作	トランザクションの範囲	同時実行制御
ダッシュボードのスナップショット (Q1～Q6、FN-017)	1 つの明示的なトランザクション、BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY ... COMMIT (DEC-015)	PostgreSQL の REPEATABLE READ は、トランザクション内のすべての文に最初の文の時点で取った同じスナップショットを与える。そのため 7 つの文すべてが production_orders の 1 つの一貫した状態を見て、003_BD の相互確認が成り立つ：タイルの合計は総数に等しく、作業量の棒の合計は Draft + In progress に等しい。読み取り専用のトランザクションはこの分離レベルで直列化の失敗を起こさないで、リトライの経路はない。ACCESS SHARE 以上のロックを取らず、書き込みを妨げない
製造指示の完了 (SCR-001 の保存、FN-021)	001_DD-FN UpdateAsync の既存の 1 回の SaveChangesAsync	変更なし：xmin による楽観的同時実行制御のチェック (WI-002 DEC-010、DEC-014)。completed_at_utc は同じ UPDATE のもう 1 つの列なので、完了日時とステータスの変更は一緒にコミットされるか、どちらもされない

トランザクションの代わりに CTE を使う 1 つの文にしない理由：同じ一貫性は得られるが、形の違う 7 つの結果（グループ化した件数、2 つの行のリスト、順位、1 行、週ごとの系列）を 1 つの結果セットにまとめることになり、EF Core は無理をしないと対応付けられない。読み取り専用のスナップショットのトランザクションなら、各クエリを単独で読みやすく、テストしやすく保てる。

デモ用の初期データ

DEC-004（ユーザー）は、納品実績のウィジェットにデータがあるよう、初期データのデモ用の製造指示にもっともらしい完了日時を付けることを求める。WI-003 の初期データだけではそれではできない。80 件のうち Completed は 16 件だけで、初期データのどの製造指示も初期データの実行の最大 49 日前に作成されており、完了は作成より前にはなりえない。そのため、その 16 件で埋められるのは推移の 12 週のうち最後の 7 週まで、週に約 2 件である。そこで DEC-013（ユーザー）は両方を行う：既存の 16 件の完了済みの製造指示の日付を付け直し、40 件の過去の完了済みの製造指示を追加する。DEC-014 は、WI-003 の初期データ（納期は最大 40 日先）では確実に届かない最後の 2 つの週の棒と「それ以降」の棒を埋めるため、十分先に納期がある未完了の製造指示を 4 件追加する。

002_DB（WI-003 DEC-011）と同様に、すべての日付はマイグレーションを実行した時点からの相対である。それ以外 — id、製品、数量、日数のオフセット — はすべて定数。

日付を付け直す既存の完了済みの製造指示（16 件）

初期データの行 seq 6、12、18、24、30、31、37、43、49、55、61、67、68、73、74、80。WI-003 の初期データの id の接頭辞とステータス Completed で識別する。製品、数量、納期は変更しないので、Screen B の納期の絞り込みと納期遅れの印には影響しない。マイグレーションは completed_at_utc = now() - c 日、updated_at_utc = completed_at_utc （完了の保存が製造指示の最後の更新）、created_at_utc = completed_at_utc - 1 日 を設定する。

seq	納期（実行日からの日数、変更なし）	c 日前に完了	リードタイム / 日	何日前に作成	期限内？
74	+0	0	9	9	期限内
43	+5	1	12	13	期限内
24	- 6	2	15	17	遅延
49	- 7	3	11	14	遅延
80	- 10	4	18	22	遅延
18	+10	6	8	14	期限内
37	+11	8	14	22	期限内
12	+16	10	21	31	期限内
73	+15	12	10	22	期限内
31	+17	14	16	30	期限内
67	+21	17	13	30	期限内
6	+22	20	19	39	期限内
61	+27	23	9	32	期限内
55	+33	26	22	48	期限内
30	+38	29	12	41	期限内
68	+40	33	17	50	期限内

納期のかなり前に完了した製造指示はもっともらしく — 期限内である（003_BD D-07）。

新しい過去の完了済みの製造指示（40 件）

すべて Completed、備考なし。それぞれ completed_at_utc = now() - c 日、created_at_utc = completed_at_utc - 1 日、updated_at_utc = completed_at_utc、due_date = 完了の工場ローカルの日付 + δ 日（負の δ = 遅れて完了）。

#	製品	数量	c 日前に完了	リードタイム / 日	完了に対する納期 δ	期限内？
1	P-1001	320	0	11	+3	期限内
2	P-1002	540	2	14	-2	遅延
3	P-1003	150	3	9	+4	期限内
4	P-1004	880	5	16	+1	期限内
5	P-1005	95	7	12	+0	期限内
6	P-1006	1,200	9	20	-3	遅延
7	P-1007	430	11	8	+5	期限内
8	P-1008	260	13	15	+2	期限内
9	P-1009	710	15	10	-1	遅延
10	P-1010	55	16	18	+6	期限内
11	P-1011	1,900	19	13	+3	期限内
12	P-1012	340	21	9	+1	期限内
13	P-1013	620	24	22	-4	遅延
14	P-1014	180	25	11	+2	期限内
15	P-1015	2,500	27	14	+7	期限内
16	P-1016	75	30	17	+0	期限内
17	P-1017	960	32	12	-2	遅延
18	P-1018	410	35	19	+4	期限内
19	P-1019	130	37	10	+1	期限内
20	P-1020	1,450	40	24	-5	遅延
21	P-1021	290	42	13	+3	期限内
22	P-1022	820	45	16	+2	期限内
23	P-1023	65	47	11	-1	遅延
24	P-1024	1,100	50	20	+5	期限内
25	P-1025	370	53	14	+1	期限内

#	製品	数量	c 日前に完了	リードタイム / 日	完了に対する納期 δ	期限内？
26	P-1026	2,200	56	9	-3	遅延
27	P-1027	145	58	18	+2	期限内
28	P-1028	690	61	12	+4	期限内
29	P-1029	310	64	21	-2	遅延
30	P-1030	1,750	66	15	+3	期限内
31	P-1001	85	69	10	+1	期限内
32	P-1002	560	71	17	-6	遅延
33	P-1003	240	74	13	+2	期限内
34	P-1004	1,300	76	22	+5	期限内
35	P-1005	420	78	11	+0	期限内
36	P-1006	95	80	16	-3	遅延
37	P-1007	780	82	14	+2	期限内
38	P-1008	210	83	19	+4	期限内
39	P-1009	1,600	85	12	-1	遅延
40	P-1010	330	88	20	+3	期限内

新しい納期が先の未完了の製造指示（4 件、DEC-014）

#	製品	数量	ステータス	納期（実行日からの日数）	何日前に作成	実行日での棒
41	P-1011	600	Draft	+49	5	第 7 週（どの実行曜日でも）
42	P-1017	1,800	Draft	+56	8	それ以降
43	P-1024	450	InProgress	+63	12	それ以降
44	P-1003	950	Draft	+42	3	第 6 週（どの実行曜日でも）

`updated_at_utc = created_at_utc`、`completed_at_utc` は `NULL`。第 k 週は $W(T) + 7k \dots W(T) + 7k + 6$ で、実行曜日 $wd = 0$ （月）... 6（日）では $T + 7k - wd \dots T + 7k + 6 - wd$ となる。そのため $T + 42$ は常に第 6 週に、 $T + 49$ は第 7 週に入り、 $T + 56$ 以降は常に「それ以降」になる。月曜または火曜の実行では WI-003 の初期データが第 6 週を空にするため、#44 が必要になる。

初期データが生む結果

上の行を、各曜日の実行、および月の 1 日・15 日・28 日の実行でシミュレーションし、実行日にダッシュボードを見た場合で確認した：

数値	結果
完了の推移（D-08）	どの実行曜日でも、12 週すべてに少なくとも 2 件の完了がある。週に約 3 件から、直近の週の 7〜10 件へと増える
期限内、直近 30 日（D-07）	どの実行曜日でも 30 件中 23 件 — 77%
平均リードタイム、直近 30 日（D-09）	30 件に基づき 13.7 日
今週の完了（D-05）	月曜の実行で 2 件、日曜の実行で 10 件まで増える — 空になることはない
今月の完了（D-06）	月の 1 日に 2 件、月の後半には 29 件まで
ステータスのタイル	124 件：35 件 Draft、25 件 InProgress、56 件 Completed、8 件 Cancelled
作業量（D-03）	納期遅れ 15。どの実行曜日でも、実行日には第 0〜7 週と「それ以降」のすべての棒が 0 でない。棒の合計は 60（35 + 25）
納期遅れ／納期間近（D-01、D-02）	実行日に納期遅れ 15 件（10 件を表示 + "Showing 10 of 15"）、納期間近 8 件
上位の製品（D-04）	27 製品が未完了数量を持つので、10 件のリストは埋まる

WI-003 の納期遅れの印と同じく、初期データの実行後に日が経つと数値はずれていく。Compose のボリュームを作り直すと、初期データは新しい日付で再実行される。

保護、指示番号、範囲

- 日付の付け直しは、WI-003 の初期データの id の接頭辞を持ち、ステータスがまだ Completed の行だけを変更する。WI-003 の初期データが実行されなかったデータベースでは該当行がなく、このステップは何もしない。
- 挿入は、WI-003 の初期データの行が存在し（デモ用のデータベースであることを示す）、44 個の新しい id がまだどれも存在しないときだけ実行する。再適用しても何も起こらず、実際の製造指示のデータベースにデモ用の履歴が加わることはない。
- id は固定の UUIDv7 の形の定数で、独自の接頭辞 0197e4a0-0000-7000-8002-... を持ち、WI-003 の ...-8001-... と区別されるので、それぞれの初期データを単独で削除できる。
- 指示番号は固定しない。ユーザーが WI-003 の初期データの後にすでに製造指示を登録している（00081 以降を使っている）可能性があるため、新しい行は初期データの年のカウンター次の 44 個の連番を使う。その後、同じマイグレーションの中でカウンターを最後の番号まで進めるので、Screen A はその後から続ける。API で最初に登録した製造指示が 00081 であることを確認する WI-003 の結合テストは 00125 に変わる。
- 指示の年：WI-003 の初期データと同じく、新しい行はすべて実行日の工場の年を使う。一部は最大 108 日前に作成されたものでもそうである。そのため初期データを 1〜4 月に実行すると、一部のデモ用の製造指示は作成日より後の年を持つ。これは見た目だけの問題で、デモ用の製造指示に 2 つのカウンターを持たせるのではなく許容する。

- **環境**：WI-003 の初期データと同じく、WI-003 の初期データが実行されたすべてのデータベース（結合テスト用のデータベースを含む）に入る。ダッシュボードの結合テストは、初期データの日付から遠くに固定した工場の時計の下で自分が作成した製造指示で確認するか、差分で確認しなければならず、初期データの合計を確認してはならない。方法は 003_DD で定める。

これらの値は実行時に計算するため、002_DB と同じ理由で、初期データは生 SQL（`migrationBuilder.Sql`）とする。

アプリのデータベース権限

変更なし。ダッシュボードには `production_orders` と `products` の `SELECT` が必要で、Screen A の保存には新しい列の `UPDATE` が必要である。`pmai_app` はすでに `production_orders` のテーブル単位の `SELECT`、`INSERT`、`UPDATE` を持っており（001_DB）、テーブル単位の権限付与は後で追加された列にも及ぶ。`DELETE` と DDL は不要で、付与もしない。下の 3 つのマイグレーションはすべて所有者として実行する。

API/DD との対応

DD の項目名は 003_BD §3 の変数名。API の項目名は 003_DD-API で確定する。

テーブル.列／クエリ	DD の項目	API の項目
Q1 の <code>status</code> ごとの件数	<code>statusCounts.*</code> （項目 7～11）	<code>statusCounts.total</code> 、 <code>.draft</code> 、 <code>.inProgress</code> 、 <code>.completed</code> 、 <code>.cancelled</code>
Q3a の行／ <code>total</code>	<code>overdue</code> （項目 16、17）	<code>overdue.total</code> 、 <code>overdue.orders[]</code>
Q3b の行／ <code>total</code>	<code>dueSoon</code> （項目 18、19）	<code>dueSoon.total</code> 、 <code>dueSoon.orders[]</code>
<code>production_orders.id</code> 、 <code>order_number</code> 、 <code>quantity</code> 、 <code>due_date</code> 、 <code>status</code> 、 <code>products.sku</code> 、 <code>name</code>	表示する製造指示の項目（M-11、M-12、M-15）	<code>orders[].id</code> 、 <code>.orderNumber</code> 、 <code>.quantity</code> 、 <code>.dueDate</code> 、 <code>.status</code> 、 <code>.product.sku</code> 、 <code>.product.name</code>
Q2 の区分	<code>workload</code> （項目 20）	<code>workload[]</code> ：{ <code>kind: overdue</code> \ <code>week</code> \ <code>later</code> , <code>weekStart?</code> , <code>orderCount</code> , <code>quantity</code> }、10 件
Q4 の行	<code>topProducts</code> （項目 22）	<code>topProducts[]</code> ：{ <code>product.sku</code> , <code>product.name</code> , <code>openQuantity</code> , <code>activeOrderCount</code> }
Q5 の週／月	<code>completedThisWeek</code> 、 <code>completedThisMonth</code> （項目 12、13）	<code>completedThisWeek</code> .{ <code>orderCount</code> , <code>quantity</code> }、 <code>completedThisMonth</code> .{...}
Q5 の <code>window_on_time</code> 、 <code>window_count</code>	<code>onTime</code> （項目 14）	<code>onTime.onTimeCount</code> 、 <code>onTime.completedCount</code> （率は表示用に導出し、重ねて送らない）
Q5 の <code>window_lead_days</code> 、 <code>window_count</code>	<code>leadTime</code> （項目 15）	<code>leadTime.averageDays</code> （なければ <code>null</code> ）、 <code>leadTime.orderCount</code>

テーブル列／クエリ	DD の項目	API の項目
Q6 の行	completionTrend （項目 23）	completionTrend[] : { weekStart, orderCount }、12 件
—（スナップショット時刻、工場の今日）	asOf （項目 4）	asOf （ISO 8601 UTC）、today （工場ローカルの日付）
production_orders.completed_at_utc	—	どのエンドポイントでも公開しない：Screen A は表示せず（REQ-033）、ダッシュボードはその集計だけを公開する

性能の見込み

002_DB と同じ規模の前提：年に約 15,000 件の製造指示。未完了の集合（未完了の作業量）は数百から数千の前半にとどまり、どの 12 週の期間の完了も最大約 3,500 件。デモ用のデータベースは 124 件。

クエリ	見込み
Q1 ステータス別件数	テーブル全体のシーケンシャルスキャン + 4 グループのハッシュ集計 — 10 万行あたり約 1 ミリ秒で、この件数では何年も許容範囲
Q2、Q4	未完了の部分インデックスのビットマップスキャン、未完了の作業量に対する集計
Q3a、Q3b	未完了の部分インデックスの順序付きスキャンで、10 行で止まる。ウィンドウの件数はグループ全体を必要とするが、未完了の作業量で上限がある
Q5、Q6	約 12 週の完了の部分インデックスの範囲スキャンの後に集計

したがって 1 回のダッシュボードの読み込みは、いくつかの上限のあるスキャンと 1 回の全件の件数取得（Q1）である。テーブルがおおよそ 100 万行を超えた場合（Q1 を維持するカウンターで置き換える）、または未完了の作業量が履歴に対して小さくなった場合は見直す。

マイグレーションの影響と復旧の限界

この順で 3 つのマイグレーション：

1. **AddProductionOrderCompletionTracking** — 列、補完、制約（1 つのトランザクション）。
- **マイグレーションの種類**：追加の後にデータの変換（補完） — expand/contract の変更の expand のステップ（`ai/rules/database.md`）。何も置き換えないので、後で contract するものはない。
 - **ステップ**：`ALTER TABLE production_orders ADD COLUMN completed_at_utc timestampz NULL`。既定値のない NULL 許容の列はカタログのみの変更で、テーブルの書き換えはない。次に `UPDATE production_orders SET completed_at_utc = updated_at_utc WHERE status = 'Completed'`（DEC-004）：完了した製造指示では、最後の更新は完了の保存か、その後の備考や納期の編集なので、これが手に入る最良の根拠で、`created_at_utc` より前になることはない。その後に両方のチェック制約。

- **ロック**： 制約は同じトランザクションの中で検証付きで追加し、`ALTER` がすでに持っている `ACCESS EXCLUSIVE` のロックの下でテーブルをスキャンする。上の規模ではそのスキャンは 1 秒よりずっと短い。はるかに大きなテーブルでは、制約を `NOT VALID` で追加し、より弱いロックで済む別のマイグレーションで検証する。
- **デプロイ順序**： マイグレーション済みのスキーマで誰かが製造指示を完了できるようになる前に、`completed_at_utc` を設定するバージョンのバックエンドが動いていなければならない。マイグレーション 1 の後に古いバックエンドが製造指示を完了すると `ck_..._matches_status` に違反し、その保存は数値を黙って壊すのではなくサーバーエラーで失敗する。既存の Compose の手順はすでにバックエンドを止めてマイグレーションを実行し、その後に新しいイメージを起動するので（`deploy/README.md`）、そこにその隙間はない。
- **副作用**： 補完は完了した各行を更新し、その `xmin` を変える。そのため、完了した製造指示についてすでに開いている Screen A の編集フォームは、保存時に通常の古いデータでの保存の競合（MSG-E009）になり、再読み込みが必要になる。これは無害で、完了した製造指示だけに当てはまる。
- **データ復旧の限界**： 戻すと列を削除するので、マイグレーション後に記録したすべての完了日時が失われる。補完と同じように `updated_at_utc` からおおよそ再構築する以外に復元できない。
- **ロールバック計画**： 両方の制約を削除してから列を削除する。その後、古いバックエンドは変更なしで動く。

2. `AddProductionOrderDashboardIndexes` — 2 つの部分インデックス。

- **マイグレーションの種類**： 追加のみ。002_DB のインデックスのマイグレーションとまったく同じく、それぞれ `migrationBuilder.Sql(..., suppressTransaction: true)` による `CREATE INDEX CONCURRENTLY IF NOT EXISTS`。
- **アトミックでない**： 途中で失敗すると `INVALID` のインデックスが残り、再試行の前に削除（`DROP INDEX CONCURRENTLY`）しなければならない。`SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;` で確認する。
- **データ復旧の限界**： なし — データを書き込まない。
- **ロールバック計画**： 両方を `DROP INDEX CONCURRENTLY IF EXISTS` で削除する。戻してもダッシュボードが遅くなるだけで、誤りにはならない。

3. `SeedDashboardDemoHistory` — デモ用の履歴（1 つのトランザクション）。

- **マイグレーションの種類**： データの変換と挿入、保護付き（上の保護を参照）。最大 16 行の初期データの日付を付け直し、最大 44 行を挿入し、初期データの年のカウンターを進める。
- **データ復旧の限界**： 日付を付け直した行の元の `created_at_utc`、`updated_at_utc`、`completed_at_utc` の値 — それ自体が以前の実行日からの相対のデモの値 — は上書きされ、戻しても復元されない。実際の製造指示は変更しない。
- **ロールバック計画**： `DELETE FROM production_orders WHERE id::text LIKE '0197e4a0-0000-7000-8002-%'` で挿入した 44 行を削除する。カウンターは進めたままにし、指示番号が再利用されることはない。日付を付け直した 16 行は新しいタイムスタンプのまま。
- **Screen B への影響**： デモ用のデータベースは 80 件から 124 件の製造指示に増える。Screen B に新しい動作はないが、初期データの数値を述べる WI-003 の文書とテスト — 002_DB の初期データの表、80 行の合計・ステータスの分布・次の番号 `00081` を確認するテスト — は計画リビジョン 2 で新しい数値に更新する。初期データの未完了の製造指示の納期は変わらないので、相対の日付を確認するテストは影響を受けない。

未決定事項

決定事項	選択肢	推奨	状態
完了の記録：列か履歴テーブルか	列／履歴テーブル	列	決定 — <code>work-items/WI-004/decisions.md</code> DEC-003
既存の完了済みの製造指示の補完	初期データ + <code>updated_at_utc</code> からの補完／NULL のまま	初期データ + 補完	決定 — DEC-004
デモ用の履歴：製造指示の追加、既存の日付の付け直し、または変換	約 40 件を追加／16 件の日付を付け直し／一部を変換／追加と付け直しの両方	両方	決定 — DEC-013（ユーザー）
初期データで 8 週先の読みより先に納期がある未完了の製造指示	4 件追加／なし	4 件追加	決定 — DEC-014（Claude。003_DB のレビューでユーザーの異議があれば見直す）
ダッシュボードが 1 つの一貫したスナップショットを得る方法	<code>REPEATABLE READ READ ONLY</code> のトランザクション／1 つの CTE の文／保証なし	トランザクション	決定 — DEC-015
完了のルールをデータベースで強制するか	2 つの CHECK 制約／アプリのみ	制約	決定 — ここに記録（どちらのルールも時計に関係なく成り立つ。制約を参照）
ステータス別件数のインデックス	なし／ステータスのインデックス／維持するカウンター	約 100 万行まではなし	決定 — ここに記録